

BCSE202L- Data Structures and Algorithm

Module 3

Dr. G. Praveen Kumar
Assistant Professor, Gr2
SMEC, SCOPE

Module:1	Algorithm Analysis	8 hours
Importance of algorithms and data structures - Fundamentals of algorithm analysis: Space and time complexity of an algorithm, Types of asymptotic notations and orders of growth - Algorithm efficiency – best case, worst case, average case - Analysis of non-recursive and recursive algorithms - Asymptotic analysis for recurrence relation: Iteration Method, Substitution Method, Master Method and Recursive Tree Method.		
Module:2	Linear Data Structures	7 hours
Arrays: 1D and 2D array- Stack - Applications of stack: Expression Evaluation, Conversion of Infix to postfix and prefix expression, Tower of Hanoi – Queue - Types of Queue: Circular Queue, Double Ended Queue (deQueue) - Applications – List: Singly linked lists, Doubly linked lists, Circular linked lists- Applications: Polynomial Manipulation.		
Module:3	Searching and Sorting	7 hours
Searching: Linear Search and binary search – Applications. Sorting: Insertion sort, Selection sort, Bubble sort, Counting sort, Quick sort, Merge sort - Analysis of sorting algorithms.		
Module:4	Trees	6 hours
Introduction - Binary Tree: Definition and Properties - Tree Traversals- Expression Trees:- Binary Search Trees - Operations in BST: insertion, deletion, finding min and max, finding the k^{th} minimum element.		

Module:5 Graphs	6 hours
Terminology – Representation of Graph – Graph Traversal: Breadth First Search (BFS), Depth First Search (DFS) - Minimum Spanning Tree: Prim's, Kruskal's - Single Source Shortest Path: Dijkstra's Algorithm.	
Module:6 Hashing	4 hours
Hash functions - Separate chaining - Open hashing: Linear probing, Quadratic probing, Double hashing - Closed hashing - Random probing – Rehashing - Extendible hashing.	
Module:7 Heaps and AVL Trees	5 hours
Heaps - Heap sort- Applications -Priority Queue using Heaps. AVL trees: Terminology, basic operations (rotation, insertion and deletion).	
Module:8 Contemporary Issues	2 hours

Text Book

1. Mark A. Weiss, Data Structures & Algorithm Analysis in C++, 4th Edition, 2013, Pearson Education.

Reference Books

1. Alfred V. Aho, Jeffrey D. Ullman and John E. Hopcroft, Data Structures and Algorithms, 1983, Pearson Education.
2. Horowitz, Sahni and S. Anderson-Freed, Fundamentals of Data Structures in C, 2008, 2nd Edition, Universities Press.
3. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Context :

Importance of algorithms and data structures - Fundamentals of algorithm analysis: Space and time complexity of an algorithm, Types of asymptotic notations and orders of growth

Queue



Meta is offering hefty compensation packages to lure top talent, the latest being a staggering Rs 2,196 crore compensation, made personally by Meta CEO Mark Zuckerberg to 24-year-old AI researcher Matt Deitke.

Experience



AI Researcher

Meta

Jul 2025 - Present · 2 months

Menlo Park, California, United States

Meta Superintelligence Lab



Co-Founder

Vercel

Nov 2024 - Jul 2025 · 9 months

Seattle, Washington, United States



AI Research & Engineering

Allen Institute for AI (AI2)

Sep 2019 - Oct 2024 · 5 years 2 months

Education



University of Washington

Doctor of Philosophy - PhD · Computer Science

2023 - 2024



University of Washington

Bachelor of Science - BS · Computer Science

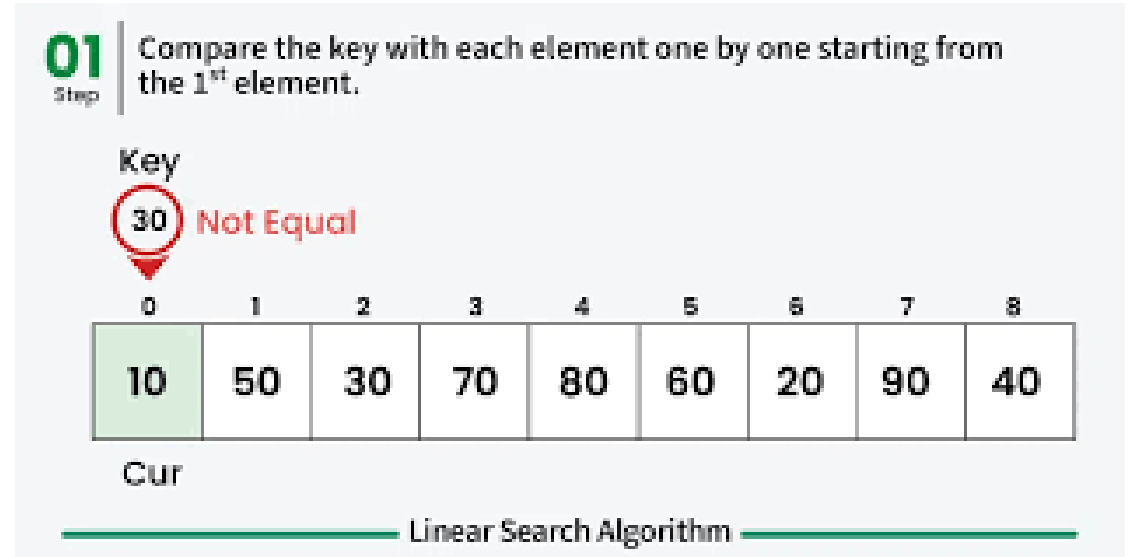
2019 - 2023

Module 3 : Searching: Linear Search and binary search

Linear search/ Sequential search is the simplest searching algorithm where each element in the list is checked one by one until the desired element is found or the list ends.

How it works:

- 1.Start from the first element of the array.
- 2.Compare each element with the target.
- 3.If a match is found, return its index.
- 4.If the end of the array is reached without a match, return -1.



Module 3 : Searching: Linear Search and binary search

```
#include <stdio.h>
```

```
int linearSearch(int arr[], int n, int key) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == key)  
            return i; // Found  
    }  
    return -1; // Not found  
}
```

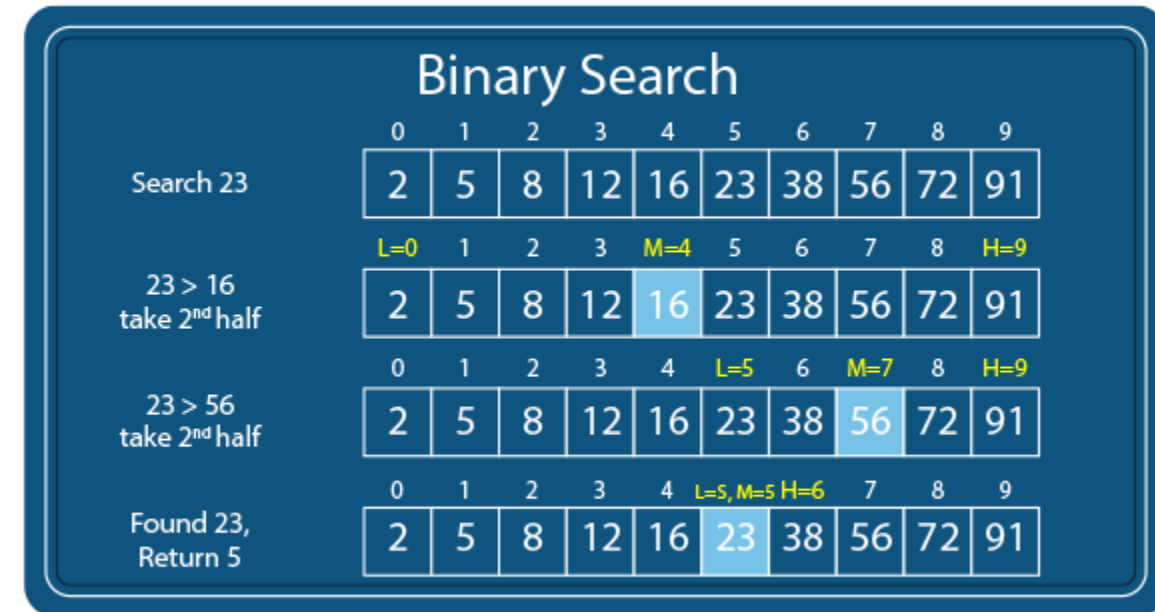
```
int main() {  
    int a[] = {15, 5, 20, 35, 2, 42, 67, 17}; // Array  
    int n = 8;                               // Size of the array  
    int data = 42;                             // Element to be searched  
    int found = 0;                             // Flag to track if element is found  
  
    for (int i = 0; i < n; i++) {  
        if (a[i] == data) {  
            printf("Element found at index: %d\n", i);  
            found = 1;  
            break;  
        }  
    }  
  
    if (found == 0) {  
        printf("Element not found\n");  
    }  
  
    return 0;  
}
```

Module 3 : Searching: Linear Search and binary search

Binary search is an efficient algorithm to find an element in a **sorted** array by repeatedly dividing the search interval in half.

How it works:

- Find the middle element of the array.
- If it matches the target, return its index.
- If the target is less, repeat the search in the left half.
- If the target is greater, repeat the search in the right half.
- Continue until the element is found or the range becomes invalid.



Module 3 : Searching: Linear Search and binary search

```
int binarySearch(int arr[], int low, int high, int key) {  
    while (low <= high) {  
        int mid = (low + high) / 2;  
  
        if (arr[mid] == key)  
            return mid; // Found  
        else if (arr[mid] < key)  
            low = mid + 1;  
        else  
            high = mid - 1;  
    }  
    return -1; // Not found  
}
```

Module 3 : Searching: Linear Search and binary search

What will the function return for arr = [3, 5, 7, 9], n = 4, key = 7?

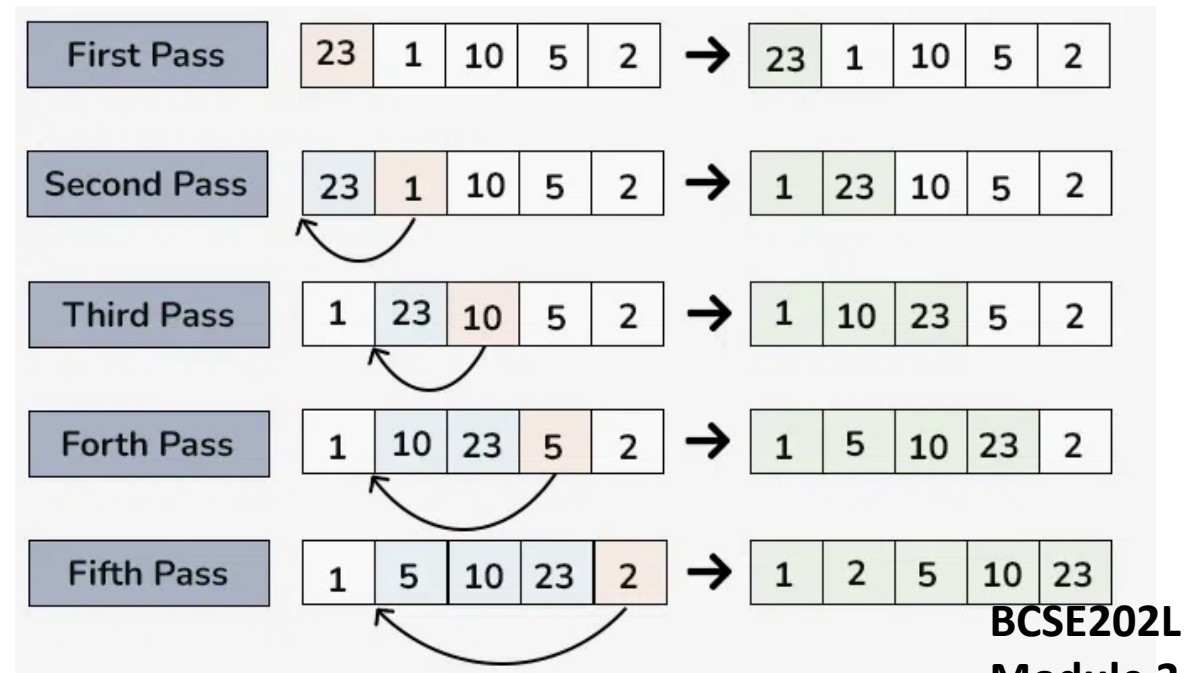
- A. 1 B. 2 C. 3 D. -1

Module 3 : Sorting: Insertion sort

Insertion Sort is a simple comparison-based **sorting algorithm** that builds the **final sorted array one element at a time**.

How it works

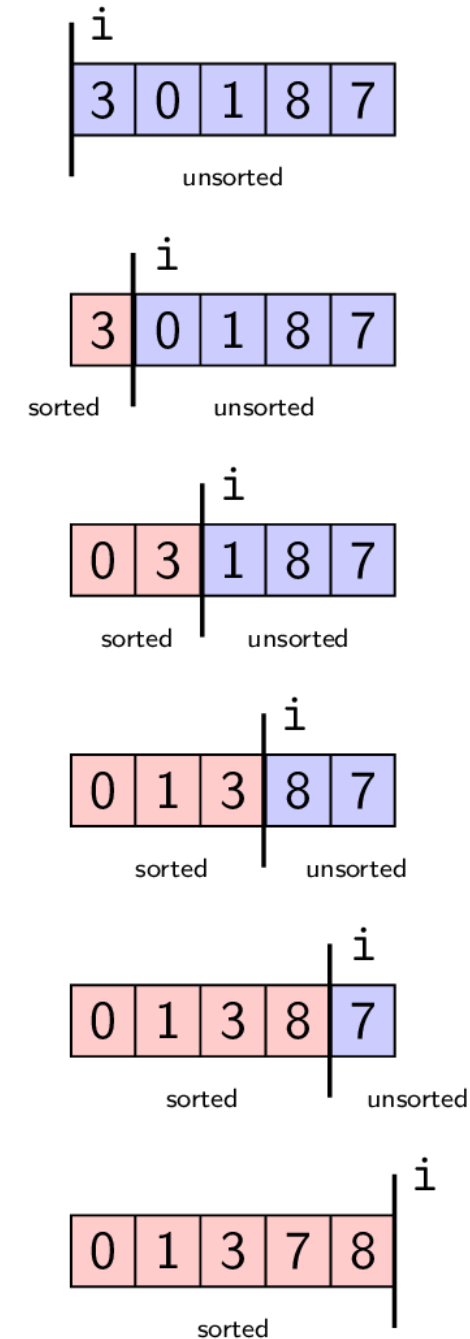
- Start from the second element (since the first element is trivially sorted).
- Compare the current element with the elements before it.
- Shift the greater elements one position ahead to make space.
- Insert the current element into its correct position in the sorted part.



Module 3 : Sorting: Insertion sort

```
for (i = 1; i < n; i++) {  
    key = arr[i];  
    j = i - 1;  
  
    // Move elements of arr[0..i-1], that are greater than key,  
    // to one position ahead of their current position  
    while (j >= 0 && arr[j] > key) {  
        arr[j + 1] = arr[j];  
        j = j - 1;  
    }  
    arr[j + 1] = key;  
}
```

Solve 5,4,10,1,6,2



Module 3 : Sorting: Insertion sort

What will be the state of the array after the second pass of insertion sort on this input? [9, 5, 1, 4]

- A. [5, 9, 1, 4]
- B. [1, 5, 9, 4]
- C. [5, 1, 9, 4]
- D. [9, 5, 1, 4]

2. How many swaps are performed by insertion sort on the array [4, 3, 2, 1]?

- A. 3
- B. 6
- C. 10
- D. 0

3. What would be the state of the array after 3 iterations of insertion sort on [7, 2, 4, 3, 5]?

- A. [2, 4, 7, 3, 5]
- B. [2, 3, 4, 7, 5]
- C. [2, 4, 3, 7, 5]
- D. [2, 3, 7, 4, 5]

4.

Module 3 : Sorting: Selection sort

Selection Sort is a simple comparison-based sorting algorithm. It works by repeatedly selecting the smallest (or largest) element from the unsorted part of the array and placing it at the beginning.

How it works

1. Divide the array into two parts:
 - Sorted part (starts empty)
 - Unsorted part (initially the full array)
2. Repeatedly:
 - Find the minimum element in the unsorted part.
 - Swap it with the first unsorted element.
 - Expand the sorted part by one element.

Key Characteristics:

- **Time Complexity:** $O(n^2)$ in all cases (best, average, worst)
- **Space Complexity:** $O(1)$ (in-place sorting)

Advantages:

- Simple to implement
- Performs well on small lists
- Minimal memory usage (only requires $O(1)$ additional space)

Disadvantages:

- Inefficient on large lists
- Doesn't take advantage of existing order in the input

Module 3 : Sorting: Selection sort

```
#include <stdio.h>
```

```
// Function to perform selection sort
```

```
void selectionSort(int arr[], int n) {
```

```
    int i, j, min_idx;
```

```
    // One by one move boundary of unsorted subarray
```

```
    for (i = 0; i < n-1; i++) {
```

```
        // Find the minimum element in unsorted array
```

```
        min_idx = i;
```

```
        for (j = i+1; j < n; j++) {
```

```
            if (arr[j] < arr[min_idx]) {
```

```
                min_idx = j;
```

```
            }
```

```
        }
```

```
    // Swap the found minimum element with the first element
```

```
    if (min_idx != i) {
```

```
        int temp = arr[min_idx];
```

```
        arr[min_idx] = arr[i];
```

```
        arr[i] = temp;
```

```
    }
```

```
}
```

```
// Function to print an array
```

```
void printArray(int arr[], int size) {
```

```
    int i;
```

```
    for (i = 0; i < size; i++) {
```

```
        printf("%d ", arr[i]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

Module 3 : Sorting: Selection sort

Original array: 64 34 25 12 22 11

After Pass 1: 11 34 25 12 22 64

After Pass 2: 11 12 25 34 22 64

After Pass 3: 11 12 22 34 25 64

After Pass 4: 11 12 22 25 34 64

After Pass 5: 11 12 22 25 34 64

Sorted array: 11 12 22 25 34 64

Module 3 : Sorting: Bubble sort

Bubble Sort is one of the simplest sorting algorithms that works by repeatedly swapping adjacent elements if they are in the wrong order

- Working Principle:**

Like bubbles rising to the surface in water, the largest element "bubbles" to the top (end of the list) in each pass.

- How it works:
- Compare **arr[j]** and **arr[j+1]**
- If they are out of order → swap them
- Repeat for **n-1** passes (outer loop)
- In each pass, one largest element gets fixed at the end

Module 3 : Sorting: Bubble sort

Array: 5 3 4 1 2

Pass 1:

Compare and swap where needed → 3 4 1 2 5 (5 is fixed at the end)

Pass 2:

→ 3 1 2 4 5 (4 fixed)

Pass 3:

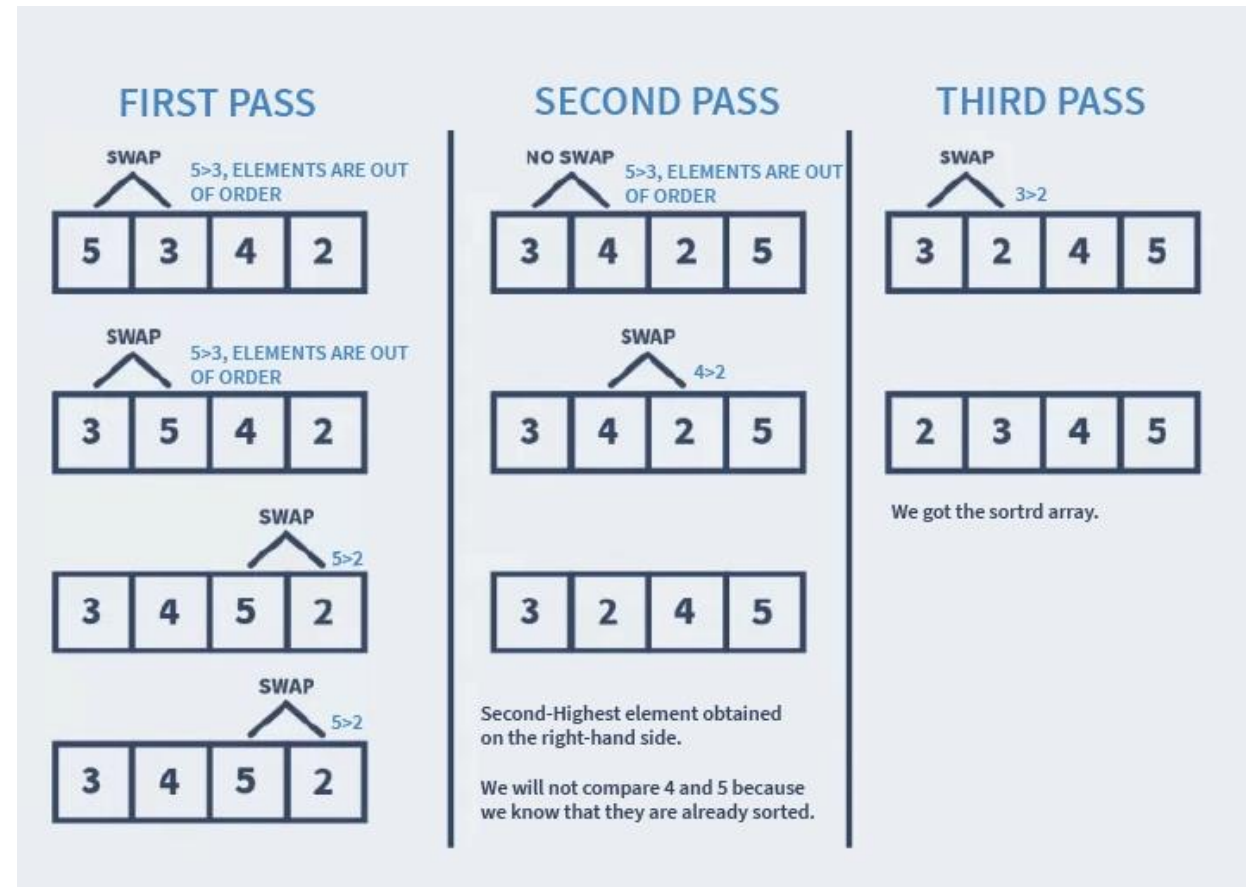
→ 1 2 3 4 5 (sorted)

Pass 4:

→ No swaps → Stop early (optimization)

Time Complexity:

- Worst-case: $O(n^2)$ - when the array is reverse sorted
- Best-case: $O(n)$ - when the array is already sorted (with optimized version)



Module 3 : Sorting: Bubble sort

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    int i, j, temp;
    int swapped;
    for (i = 0; i < n - 1; i++) {
        swapped = 0; // optimization flag

        // Last i elements are already in place
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap arr[j] and arr[j+1]
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }
        // If no elements were swapped, array is sorted
        if (swapped == 0)
            break;
    }
}
```

```
int main() {
    int arr[] = {5, 3, 4, 1, 2};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    bubbleSort(arr, n);

    printf("\nSorted array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

Module 3 : Sorting: Bubble sort

```
void bubbleSort(int ar[])
```

```
{  
    for (int i = (ar.length - 1); i >= 0; i--)  
    {  
        for (int j = 1; j <= i; j++)  
        {  
            if (ar[j-1] > ar[j])  
            {  
                int temp = ar[j-1];  
                ar[j-1] = ar[j];  
                ar[j] = temp;  
            }  
        }  
    }  
}
```

$i=n$
 $\sum_{i=0} O(i)$

$$\sum_{i=0}^{i=n} O(i) = 1 + 2 + 3 + \dots + (n-1) = O(n^2)$$

Module 3 : Sorting: Bubble sort

1. What is the primary operation performed in Bubble Sort?

- A) Dividing the array into halves
- B) Comparing and swapping adjacent elements
- C) Merging sorted subarrays
- D) Searching for the maximum element

2. How many passes are required to sort an array of 5 elements using Bubble Sort in the worst case?

- A) 2
- B) 3
- C) 4
- D) 5

3. In Bubble Sort, after the first pass on the array [5, 3, 8, 4, 2], what is the state of the array?

- A) [2, 3, 4, 5, 8]
- B) [3, 5, 4, 2, 8]
- C) [5, 3, 4, 2, 8]
- D) [3, 4, 2, 5, 8]

Module 3 : Sorting: Bubble sort

4. Which of the following arrays will require the minimum number of passes in Bubble Sort?

- A) [5, 4, 3, 2, 1] B) [1, 2, 3, 4, 5] C) [1, 5, 2, 4, 3] D) [4, 1, 5, 2, 3]

5. What happens in the last pass of Bubble Sort on an array of n elements?

- A) The largest element is placed at the start B) No swaps occur if the array is sorted
C) The smallest element is placed at the end D) The array is divided into two halves

6. What will be the array after the second pass of bubble sort on [4, 2, 5, 1]?

- A. [2, 1, 4, 5] B. [2, 4, 1, 5] C. [1, 2, 4, 5] D. [4, 1, 2, 5]

Module 3 : Sorting: Counting sort

Counting sort is a non-comparison-based sorting algorithm that works by counting the occurrences of each unique element in the input array and then arranging elements in sorted order using this count information.

- **How it works**

Instead of comparing elements (like bubble, selection, or quicksort), it:

- Finds the **range** of numbers (min to max).
- Creates a **count array** to store frequency of each number.
- Modifies the count array to store **cumulative counts** (prefix sums).
- Places elements into an **output array** in sorted order.

Module 3 : Sorting: Counting sort

```
#include <stdio.h>
#include <stdlib.h>
```

```
// Counting sort for non-negative integers
```

```
void countingSort(int arr[], int n) {
    if (n <= 0) return;
```

```
    // 1. Find maximum value in arr
```

```
    int max_val = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > max_val) max_val = arr[i];
```

```
    // 2. Allocate count array (size = max_val + 1) and initialize to 0
```

```
    int *count = (int *)calloc(max_val + 1, sizeof(int));
    if (!count) {
        printf("Memory allocation failed\n");
        return;
    }
```

```
    // 3. Store frequency of each value
    for (int i = 0; i < n; i++)
        count[arr[i]]++;
```

```
    // 4. Convert counts to prefix sums (cumulative counts) for
    stability
```

```
    for (int i = 1; i <= max_val; i++)
        count[i] += count[i - 1];
```

```
    // 5. Build output array (stable) by iterating input from right to
    left
```

```
    int *output = (int *)malloc(n * sizeof(int));
    if (!output) {
        free(count);
        printf("Memory allocation failed\n");
        return;
```

```
    }
    for (int i = n - 1; i >= 0; i--) {
        output[count[arr[i]] - 1] = arr[i];
        count[arr[i]]--;
    }
```

Module 3 : Sorting: Counting sort

```
// 6. Copy sorted items back to original array
for (int i = 0; i < n; i++)
    arr[i] = output[i];

free(count);
free(output);
}

int main() {
    int n;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid size\n");
        return 1;
    }
}
```

```
int *arr = (int *)malloc(n * sizeof(int));
if (!arr) {
    printf("Memory allocation failed\n");
    return 1;
}
```

```
printf("Enter %d non-negative integers (space separated):\n",
n);
for (int i = 0; i < n; i++) {
    if (scanf("%d", &arr[i]) != 1 || arr[i] < 0) {
        printf("Invalid input (must be non-negative integer)\n");
        free(arr);
        return 1;
    }
}
```

```
countingSort(arr, n);

printf("Sorted array: ");
for (int i = 0; i < n; i++)
    printf("%d ", arr[i]);
printf("\n");
```

```
free(arr);
return 0;
```